PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1) IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- **Denominación del Programa de Formación**: Tecnología en Análisis y Desarrollo de Software
- **Código del Programa de Formación:** 3234206
- **Nombre del Proyecto:** Construcción de software integrador de tecnologías orientadas a servicios
- **Fase del Proyecto:** Ejecución
- **Actividad de Proyecto:** Introducción a funciones
- **Competencia:** DESARROLLAR LA SOLUCIÓN DE SOFTWARE DE ACUERDO CON EL DISEÑO Y METODOLOGÍAS DE DESARROLLO
- **Resultados de Aprendizaje Alcanzar:** Resolver procesos lógicos a través de la implementación de Algoritmos y el lenguaje de programación seleccionado.
- **Duración de la Guía:** 5 Horas

2) PRESENTACIÓN

Aprendiz SENA:

La presente **Guía de Aprendizaje** tiene como propósito orientar al aprendiz en la comprensión de los conceptos fundamentales relacionados con las **API (Application Programming Interface)** y el manejo de **solicitudes HTTP**, elementos esenciales en el desarrollo de aplicaciones modernas y en la integración de sistemas de información.

En el contexto actual del desarrollo de software, las API se constituyen como un pilar clave para la comunicación entre aplicaciones, permitiendo el intercambio de información de manera estructurada, segura y eficiente. Por ello, resulta indispensable que el aprendiz adquiera las bases conceptuales y prácticas que le permitan comprender su funcionamiento, identificar sus componentes y aplicar correctamente los diferentes tipos de solicitudes utilizadas en entornos web.

Esta guía ha sido diseñada bajo el **enfoque de formación por competencias del SENA**, estructurada en diferentes **momentos de aprendizaje**, que facilitan la reflexión, la apropiación del conocimiento, la práctica y la transferencia a situaciones reales del entorno productivo. A lo largo del desarrollo de la guía, el aprendiz fortalecerá habilidades técnicas relacionadas con la arquitectura cliente-servidor, el uso de métodos HTTP y el análisis de respuestas de una API.

Finalmente, se espera que, mediante el desarrollo de las actividades propuestas, el aprendiz logre una comprensión integral del tema, sentando bases sólidas para el uso e implementación de servicios API en proyectos de desarrollo de software.

3) FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

A. Actividades de reflexión inicial.

En el entorno actual del desarrollo de software, las aplicaciones no funcionan de manera aislada. De forma constante, requieren **intercambiar información**, **consumir servicios externos** y **comunicarse con otros sistemas** para ofrecer soluciones completas y eficientes a los usuarios.
Seguramente, como aprendiz, has interactuado con aplicaciones que permiten iniciar sesión con una cuenta externa, consultar información en tiempo real, realizar pagos en línea o visualizar datos provenientes de diferentes fuentes. Detrás de estas funcionalidades existe una necesidad fundamental: **establecer un medio confiable de comunicación entre sistemas** que han sido desarrollados de forma independiente.

En este punto, es importante reflexionar sobre algunas preguntas clave:

- ¿Cómo puede una aplicación acceder a información que se encuentra en otro sistema?
- ¿De qué manera se garantiza que la información intercambiada sea correcta y segura?
- ¿Qué mecanismos permiten que diferentes tecnologías y plataformas se entiendan entre sí?

Estas necesidades evidencian la importancia de contar con **estructuras estandarizadas** que faciliten la comunicación entre aplicaciones, evitando reprocesos, duplicidad de información y errores en la integración de sistemas. Comprender estas problemáticas permite al aprendiz reconocer que, antes de programar una solución, es necesario **entender el contexto, la interacción entre sistemas y los retos que se buscan resolver**.

Preguntas por resolver:

1. ¿Has utilizado aplicaciones que te permitan iniciar sesión usando una cuenta de otra plataforma, como correo electrónico o redes sociales?
    a. ¿Qué crees que ocurre entre esas aplicaciones para permitir este acceso?
2. Cuando realizas un pago en línea desde una aplicación o sitio web, ¿consideras que la información del pago se procesa en el mismo sistema o intervienen otros servicios externos?
3. ¿Cómo crees que una aplicación de transporte o domicilios obtiene información en tiempo real sobre ubicación, rutas o disponibilidad de servicios?
4. En tu vida diaria, ¿qué ejemplos conoces donde una persona o entidad actúe como intermediario para facilitar una comunicación o un servicio entre dos partes?
5. ¿Qué pasaría si cada aplicación tuviera que construir desde cero funcionalidades como pagos, mapas o autenticación de usuarios?
6. ¿Por qué consideras importante que diferentes sistemas, desarrollados por empresas o equipos distintos, puedan “hablar el mismo idioma”?
7. Desde tu experiencia como usuario, ¿qué problemas podrían presentarse si las aplicaciones no tuvieran una forma organizada y segura de intercambiar información?
8. ¿Crees que es posible integrar aplicaciones desarrolladas en diferentes lenguajes o tecnologías?
    a. ¿Qué necesitarían para poder comunicarse entre sí?

Estrategias didácticas activas:

- Lluvia de ideas.
- Aprendizaje basado en experiencias
- Discusión guiada

Duración de la actividad 30 minutos.

B. Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje.

Luego de identificar, a partir de situaciones cotidianas, la necesidad de que diferentes aplicaciones y sistemas puedan intercambiar información de manera organizada surge la pregunta sobre **cómo se resuelve esta necesidad en el ámbito del desarrollo de software**.

En el contexto informático, la comunicación entre sistemas no se da de forma improvisada. Por el contrario, requiere de **mecanismos estructurados**, **reglas claras** y **estándares** que permitan que aplicaciones desarrolladas en distintos lenguajes, plataformas o entornos puedan comprenderse entre sí, manteniendo la integridad y seguridad de la información.

Es en este escenario donde el desarrollo de software propone soluciones basadas en la **arquitectura cliente-servidor**, en la cual una aplicación solicita información o servicios y otra se encarga de procesarlos y responder. Para que este intercambio sea posible, se hace necesario definir **interfaces de comunicación** que establezcan qué información se puede solicitar, cómo debe enviarse y qué tipo de respuesta se puede esperar.

Este momento de contextualización permite al aprendiz comprender que las **API** no surgen como una moda tecnológica, sino como una **respuesta a una necesidad real del entorno productivo**, orientada a facilitar la integración de sistemas, optimizar procesos y mejorar la experiencia del usuario final. A partir de aquí, se inicia el acercamiento al concepto de API como una solución informática que estructura y regula la comunicación entre aplicaciones.

Concepto de API

Una **API (Application Programming Interface)** es una **interfaz de programación** que define un conjunto de reglas, estructuras y protocolos que permiten la **comunicación controlada entre aplicaciones o sistemas de información**. A través de una API, un sistema puede solicitar servicios o información a otro, sin necesidad de conocer cómo está construido internamente.

En el desarrollo de software, las API actúan como un **punto de acceso estandarizado**, facilitando la integración entre aplicaciones, promoviendo la reutilización de servicios y garantizando que el intercambio de información se realice de forma segura, organizada y eficiente.

La petición como base de la comunicación en una API

La interacción entre sistemas mediante una API se fundamenta en el intercambio de **peticiones** y **respuestas**. Una **petición** representa la solicitud que realiza un sistema cliente a un sistema servidor para acceder a un recurso o ejecutar una acción específica.

Toda petición a una API se compone de elementos clave que permiten al servidor interpretar correctamente la solicitud:

- **Recurso o endpoint:** Es el punto específico de la API al que se desea acceder. Representa el recurso sobre el cual se realizará la operación, como usuarios, productos o registros.
- **Método de la petición:** Indica la acción que el cliente desea realizar sobre el recurso, como consultar información, enviar datos, actualizar o eliminar un registro. Este método permite al servidor comprender la intención de la solicitud.
- **Encabezados (headers):** Contienen información adicional que acompaña la petición, como el formato de los datos, credenciales de acceso o parámetros de seguridad. Los encabezados ayudan a establecer las condiciones bajo las cuales se procesa la solicitud.
- **Cuerpo de la petición (body):** Es la información que se envía al servidor cuando la petición requiere transmitir datos. Generalmente se utiliza para crear o actualizar información dentro del sistema.

Tipos de peticiones en una API

Principales verbos HTTP y su uso en el desarrollo web: En el contexto del desarrollo web, las API utilizan el **protocolo HTTP** como medio de comunicación entre el cliente y el servidor. Este protocolo define un conjunto de **verbos o métodos HTTP** que permiten indicar de forma clara **qué acción se desea realizar** sobre un recurso.

Cada verbo cumple un propósito específico y su uso adecuado facilita la claridad, el mantenimiento y la escalabilidad de las aplicaciones.

1. **GET – Consulta de información:** El método GET se utiliza para solicitar información al servidor sin modificar los datos existentes. Es el verbo más común y se emplea cuando se requiere consultar uno o varios recursos.

   Usos comunes en el desarrollo web:

   - Obtener listas de información (usuarios, productos, registros)
   - Consultar el detalle de un recurso específico
   - Cargar información en vistas o interfaces de usuario

   Características clave:

   - No altera la información del servidor
   - Permite la lectura de datos
   - Es utilizado frecuentemente para mostrar información

2. **POST – Envío y creación de información**: El método **POST** se utiliza para **enviar información al servidor**, generalmente con el propósito de crear un nuevo recurso o ejecutar una acción que implica procesamiento de datos.

   Usos comunes en el desarrollo web:

   - Registrar nuevos usuarios
   - Crear pedidos, formularios o registros
   - Enviar información desde formularios web

   Características clave:

   - Incluye un cuerpo con datos
   - Genera cambios en el servidor
   - No es idempotente (cada solicitud puede generar un resultado distinto)

3. **PUT – Actualización completa de información:** El método **PUT** se emplea para **actualizar completamente un recurso existente**. El cliente envía toda la información del recurso, incluso aquella que no ha cambiado.

   Usos comunes en el desarrollo web:

   - Actualizar perfiles completos
   - Reemplazar información de un registro existente

   Características clave:

   - Reemplaza el recurso completo
   - Requiere enviar todos los datos del recurso
   - Es idempotente (múltiples solicitudes generan el mismo resultado)

4. **PATCH – Actualización parcial de información:** El método **PATCH** se utiliza para **actualizar parcialmente un recurso**, enviando únicamente los campos que se desean modificar.

   Usos comunes en el desarrollo web:

   - Cambiar el estado de un registro
   - Actualizar un dato específico (correo, contraseña, estado)

   Características clave:

   - Modifica solo una parte del recurso
   - Reduce el volumen de datos enviados
   - Es más eficiente que PUT en ciertos escenarios

5. **DELETE – Eliminación de información:** El método **DELETE** se utiliza para **eliminar un recurso** del servidor.

   Usos comunes en el desarrollo web:

   - Eliminar registros
   - Borrar usuarios, productos o elementos de una lista

   Características clave:

   - Remueve información del sistema
   - Puede ser lógico (marcado como inactivo) o físico
   - Requiere especial cuidado por su impacto

Importancia del uso correcto de los verbos HTTP

El uso adecuado de los verbos HTTP permite:

- Claridad en la intención de cada petición
- Mejores prácticas de desarrollo web
- APIs más predecibles y mantenibles
- Comunicación estandarizada entre sistemas

Comprender las diferencias y usos de los métodos HTTP es fundamental para el diseño y consumo de API, y constituye una base esencial para avanzar hacia la implementación práctica en proyectos de desarrollo web.

Materiales de formación:

Visual estudio code

Computador

Navegador

Estrategias didácticas activas:

- Clase magistral breve con mediación activa.
- Aprendizaje activo
- Aprendizaje significativo

Duración de la actividad 1 hora.

A. Actividades de apropiación del conocimiento.

Luego de reflexionar sobre la necesidad de comunicación entre sistemas, contextualizar el uso de las API como una solución informática y comprender los principales conceptos asociados a las peticiones y los verbos HTTP, es momento de avanzar hacia la **apropiación del conocimiento**.

En este momento, el aprendiz empieza a **relacionar los conceptos teóricos con situaciones concretas del desarrollo de software**, fortaleciendo su comprensión sobre

cómo se estructuran las peticiones a una API y cómo estas permiten interactuar con recursos de manera organizada y estandarizada.

La apropiación del conocimiento implica reconocer el papel de la **arquitectura cliente-servidor**, identificar los componentes de una petición (recurso, método, encabezados y cuerpo) y comprender las diferencias entre los principales verbos HTTP y su uso adecuado en el desarrollo web. Estos elementos constituyen la base para analizar, interpretar y construir solicitudes que respondan a necesidades reales del entorno productivo.

A partir de este momento, el aprendiz se enfocará en **comprender el funcionamiento operativo de las peticiones a una API**, analizando su estructura y comportamiento, como paso previo a la aplicación práctica y al desarrollo de soluciones que integren sistemas de información.

Actividades Prácticas:

Parte 1: Solicitudes de consulta (GET)

- **Solicitud 1:** Realice una solicitud GET para obtener la lista completa de usuarios disponibles en el servicio.
- **Solicitud 2:** Realice una solicitud GET para consultar la información de un usuario específico, utilizando su identificador.
- **Solicitud 3:** Realice una solicitud GET para obtener todas las publicaciones (posts) asociadas a un usuario determinado.

Parte 2: Creación de información (POST)

- **Solicitud 4:** Realice una solicitud POST para crear una nueva publicación asociada a un usuario existente.
  Incluya información como título y contenido.
- **Solicitud 5:** Realice una solicitud **POST** para registrar un nuevo comentario relacionado con una publicación.

Parte 3: Actualización de información (PUT y PATCH)

- **Solicitud 6:** Realice una solicitud PUT para actualizar completamente la información de una publicación existente.
- **Solicitud 7:** Realice una solicitud PATCH para modificar únicamente un campo específico de esa publicación.

Parte 4: Eliminación de información (DELETE)

- **Solicitud 8:** Realice una solicitud DELETE para eliminar una publicación existente.

Parte 5: Análisis y verificación de respuestas

- **Solicitud 9:** Repita una solicitud GET sobre el recurso eliminado o modificado y analice la respuesta obtenida.
- **Solicitud 10:** Realice una solicitud GET general y compare la estructura de la respuesta con las solicitudes anteriores, identificando cambios y comportamientos del servicio.

Durante el desarrollo de la actividad, tenga en cuenta:

1. Identificar el verbo HTTP utilizado en cada solicitud.
2. Analizar el código de estado de la respuesta.
3. Observar la estructura del cuerpo de la respuesta en formato JSON.
4. Reconocer diferencias entre las operaciones de creación, actualización y eliminación.

Resultado esperado:

1. Comprender el flujo de una operación CRUD a través de una API.
2. Utilizar correctamente los principales verbos HTTP.
3. Analizar respuestas de un servicio API.
4. Relacionar conceptos teóricos con una experiencia práctica guiada.

Materiales de formación:

- Planteamiento de los ejercicios
- Pc
- Navegador
- Visual estudio code
- Archivo db.json

Estrategias didácticas activas:

- Acompañamiento individual
- Resolución progresiva
- Aprendizaje por descubrimiento

- Debate técnico breve
- Retroalimentación continua

Evidencias de aprendizaje:

- Codigo fuente con repositorio

Duración de la actividad 1.5 horas.

B. Actividades de transferencia del conocimiento.

Después de haber comprendido la necesidad de comunicación entre sistemas, contextualizado el uso de las API como una solución informática y apropiado los conceptos y procedimientos relacionados con las peticiones HTTP y las operaciones CRUD, es momento de **transferir el conocimiento adquirido a situaciones más cercanas al entorno productivo**.

En este momento, el aprendiz pone en práctica lo aprendido para **analizar, proponer y resolver escenarios que simulan necesidades reales del desarrollo web**, demostrando su capacidad para aplicar los conceptos de API más allá de un ejercicio guiado. La transferencia del conocimiento implica reconocer cuándo y cómo utilizar una API, seleccionar correctamente los verbos HTTP y estructurar solicitudes de acuerdo con el propósito de la solución planteada.

Este espacio permite al aprendiz fortalecer su **pensamiento lógico**, su capacidad de **análisis de problemas** y su criterio técnico, al enfrentarse a situaciones donde debe tomar decisiones fundamentadas sobre el uso de servicios API, tal como ocurre en contextos reales de trabajo en el sector productivo.

A través de las actividades propuestas en este momento, el aprendiz consolidará su aprendizaje, evidenciando no solo el dominio conceptual y procedimental, sino también la capacidad de **aplicar el conocimiento de manera autónoma y contextualizada**.

Actividades de aprendizaje:

Ejercicios Prácticos

Enunciado 1: (Usuarios activos y sus publicaciones)

Una aplicación web requiere mostrar un listado de usuarios activos junto con la cantidad de publicaciones que han realizado. Sin embargo, no todos los usuarios han creado publicaciones. El sistema debe identificar correctamente estos casos.

Requerimientos

- Consultar la lista completa de usuarios.
- Consultar la lista de publicaciones.
- Identificar cuáles usuarios tienen publicaciones asociadas.
- Calcular la cantidad de publicaciones por usuario.
- Mostrar también los usuarios que no tienen publicaciones.

Datos de entrada

- Endpoint de usuarios (users).
- Endpoint de publicaciones (posts).
- Identificador del usuario (userId)

Datos de salida

- Listado de usuarios con:
    - Nombre del usuario
    - Cantidad de publicaciones asociadas (puede ser 0)

1. **Enunciado 2: (Publicaciones con y sin comentarios)**

El área de contenido necesita identificar qué publicaciones han generado interacción y cuáles no. Para ello, se requiere analizar las publicaciones y sus comentarios asociados.

Requerimientos:

- Consultar todas las publicaciones.
- Consultar todos los comentarios.
- Relacionar comentarios con sus publicaciones.
- Identificar publicaciones sin comentarios.
- Clasificar publicaciones según tengan o no comentarios.

Datos de entrada:

- Endpoint de publicaciones (posts)
- Endpoint de comentarios (comments)
- Identificador de la publicación (postId)

Datos de salida:

- Listado de publicaciones con:
    - Título
    - Número de comentarios
    - Estado: “Con comentarios” o “Sin comentarios”

Enunciado 3: (Búsqueda específica de información)

Un usuario del sistema desea consultar información puntual sobre una publicación específica y conocer si existe interacción asociada a ella.

Requerimientos:

- Consultar todas las publicaciones.
- Buscar una publicación específica por su identificador.
- Consultar los comentarios relacionados con esa publicación.
- Validar si existen o no comentarios asociados.

Datos de entrada:

- ID de la publicación a consultar
- Endpoint de publicaciones (posts)
- Endpoint de comentarios (comments)

Datos de salida:

- Información detallada de la publicación:
    - Título
    - Contenido
    - Número de comentarios asociados

Enunciado 4: (Eliminación lógica y validación de datos)

Antes de eliminar una publicación, el sistema debe validar si dicha publicación tiene comentarios asociados. Si tiene comentarios, no debe eliminarse; de lo contrario, puede proceder.

Requerimientos:

- Consultar las publicaciones.
- Consultar los comentarios.
- Verificar si una publicación específica tiene comentarios.
- Si no tiene comentarios, ejecutar la eliminación.
- Validar el resultado mediante una nueva consulta.

Datos de entrada:

- ID de la publicación
- Endpoint de publicaciones (posts)
- Endpoint de comentarios (comments)

Datos de salida:

- Mensaje de resultado:
    - “Publicación eliminada correctamente”
    - “No se puede eliminar la publicación porque tiene comentarios”

Materiales de formación:

- Editor de código (VS Code)
- Navegador web con herramientas de inspección
- Repositorio de Git
- Archivo transferencia.js

Estrategias didácticas activas:

- Resolución de problemas prácticos.
- Aprendizaje basado en casos (ABP)
- Pensamiento algorítmico
- Socialización técnica
- Trabajo autónomo con retroalimentación del instructor.
- Uso de GitHub como portafolio de evidencias.

Evidencias de aprendizaje:

- La solución de los ejercicios en archivos separados
- Comentarios en el código explicando el uso de los diferentes métodos.
- Registro de commits claros en GitHub.
- Comentarios de los análisis requeridos

Duración de la actividad 2 horas.

2. ACTIVIDADES DE EVALUACIÓN

Tome como referencia la técnica e instrumentos de evaluación citados en la guía de Desarrollo Curricular

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación

EJECUCIÓN	DESARROLLAR LA STRUCTURA DE DATOS, CONSTRUCCIÓN, CODIFICACIÓN E INTERFAZ DE USUARIO, PRUEBAS Y VALIDACIÓN DE LOS MÓDULOS DEL SISTEMA DE INFORMACIÓ	Identifica y describe, en un sistema de información dado, os datos de entrada, procesamiento de los datos e información generada, según necesidades del cliente.	Conocimiento y Producto	Solucionar problemas de lógica proposicional. Usar estructuras secuenciales en la construcción de algoritmos. Utilizar estructuras de control en la construcción de algoritmos. Emplear estructuras cíclicas en la construcción de algoritmos. Construir algoritmos con arreglos.	Listas de chequeo

3. GLOSARIO DE TÉRMINOS

API (Application Programming Interface): Interfaz de programación que permite la comunicación controlada entre aplicaciones o sistemas de información, mediante reglas y estructuras previamente definidas.

Arquitectura cliente-servidor: Modelo de comunicación en el cual un cliente solicita servicios o información a un servidor, y este último procesa la solicitud y envía una respuesta.

Asincronía: Capacidad de un programa para ejecutar operaciones sin bloquear el flujo principal de ejecución, permitiendo que otras tareas continúen mientras se espera una respuesta.

Body (Cuerpo de la petición): Parte de una solicitud HTTP que contiene los datos enviados al servidor, generalmente en formato JSON, utilizada en operaciones como POST, PUT o PATCH.

Cliente: Aplicación o sistema que realiza solicitudes a un servidor para acceder a recursos o servicios.

CRUD: Conjunto de operaciones básicas para la gestión de información: Create (crear), Read (leer), Update (actualizar) y Delete (eliminar).

Endpoint: Dirección específica dentro de una API que permite acceder a un recurso determinado.
HTTP (Hypertext Transfer Protocol): Protocolo de comunicación utilizado para el intercambio de información entre clientes y servidores en la web.

Headers (Encabezados): Información adicional enviada en una solicitud o respuesta HTTP, que define aspectos como formato de datos, autenticación y condiciones de la comunicación.

JSON (JavaScript Object Notation): Formato ligero de intercambio de datos, ampliamente utilizado en APIs por su facilidad de lectura y compatibilidad con diferentes lenguajes de programación.

json-server: Herramienta que permite simular una API REST a partir de un archivo JSON, utilizada comúnmente con fines educativos y de prototipado.

Métodos HTTP (Verbos HTTP): Acciones definidas por el protocolo HTTP que indican la operación a realizar sobre un recurso, como GET, POST, PUT, PATCH y DELETE.

Promesa (Promise): Objeto en JavaScript que representa el resultado futuro de una operación asincrónica, permitiendo manejar correctamente respuestas exitosas o errores.

Recurso: Entidad o conjunto de datos expuestos por una API, como usuarios, publicaciones o comentarios.

Request (Petición): Mensaje enviado por un cliente a un servidor solicitando un recurso o la ejecución de una acción específica.

Response (Respuesta): Mensaje enviado por el servidor al cliente como resultado de una petición, que incluye un código de estado y, opcionalmente, datos.

REST (Representational State Transfer): Estilo de arquitectura para el diseño de APIs que se basa en el uso de recursos, métodos HTTP y comunicación sin estado.

Servidor: Sistema que recibe, procesa y responde a las solicitudes realizadas por los clientes.

Status Code (Código de estado HTTP): Código numérico que indica el resultado de una solicitud HTTP, como éxito, error o falta de autorización.

4. REFERENTES BILBIOGRÁFICOS

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. University of California.
(Tesis fundamental donde se define el estilo arquitectónico REST).

Mozilla Developer Network. (s. f.). *HTTP | MDN Web Docs*.
Recurso de referencia para el estudio del protocolo HTTP, métodos, encabezados y códigos de estado.

Mozilla Developer Network. (s. f.). *JavaScript asynchronous programming*.
Documentación oficial para el manejo de asincronía, promesas y async/await en JavaScript.

JSONPlaceholder. (s. f.). *Free fake REST API*.
Servicio utilizado para la simulación y consumo de API REST en entornos de aprendizaje.

Typicode. (s. f.). *json-server*.
Herramienta para la creación de APIs REST simuladas a partir de archivos JSON, ampliamente utilizada con fines educativos.

W3C. (s. f.). *HTTP Semantics*.
Especificación oficial del protocolo HTTP y sus métodos.

Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
Referencia para el uso de JavaScript moderno, manejo de datos y asincronía.

SENA. (2019). *Lineamientos pedagógicos para la formación profesional integral*. Servicio Nacional de Aprendizaje.
Documento base para el diseño de guías de aprendizaje por competencias.

5. CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	**John Freddy Becerra Castellanos**	**Instructor Contratista**	**CIMI**	**01/02/2026**

6. CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)					